service location to obtain an instance of `ICurrencyProvider` using an injected context object.

With the FluentValidation library, you can inject the `ICurrencyProvider` directly into your validator, as shown in the following listing. This requires fewer gymnastics to get the desired functionality and makes your validator's dependencies explicit.

Listing 32.13 Currency converter validator using dependency injection

```
public class CurrencyConverterModelValidator
    : AbstractValidator<CurrencyConverterModel>
{
    public CurrencyConverterModelValidator(ICurrencyProvider provider)  ❶
    {
        RuleFor(x => x.CurrencyFrom)
            .NotEmpty()
            .Length(3)
            .Must(value => provider                                     ❷
                .GetCurrencies()                                        ❷
                .Contains(value))                                       ❷
            .WithMessage("Not a valid currency code");

        RuleFor(x => x.CurrencyTo)
            .NotEmpty()
            .Length(3)
            .MustBeCurrencyCode(provider.GetCurrencies());              ❸

        RuleFor(x => x.Quantity)
            .NotNull()
            .InclusiveBetween(1, 1000);
    }
}
```

❶ Injects the service using standard constructor dependency injection

❷ Uses the injected service in a Must() rule

❸ Uses the injected service with an extension method

The final feature I'll show demonstrates how much easier it is to write validators that span multiple properties with FluentValidation. For example,